UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026
Applied Parallel Programming

Lecture 19: GPU Systems Architecture

Volodymyr Kindratenko

4/2/2026

Course Reminders

- Labs
  - Lab 7 due this Friday
- Project
  - Project milestone 2 is due on 4/10

10	4/2/26	Lecture 19	GPU Systems Architecture
	4/3/26	Lab 7	Scan
11	4/7/26	Lecture 20	Tasks Parallelism and Asynchronous Data Transfer
	4/9/26	Lecture 21	Sparse Matrix
	4/10/26	PM2 due	
12	4/14/26	Lecture 22	Advanced Optimizations for Projects - part 2
	4/16/26	Lecture 23	Other Acceleration APIs
	4/17/26	Lab 8	Sparse Matrix Multiplication
13	4/21/26	Lecture 24	CUDA Dynamic Parallelism or Guest Lecture on Triton
	4/23/26	Lecture 25	Guest lecture by Howie Liu, Cerebras
	4/24/26	no lab/PM due	
14	4/28/26	Lecture 26	Guest lecture on cuTiles
	4/30/26	Lecture 27	Course Retrospective
	5/1/26	PM3 due	
	5/5/26	Midterm 2	

Today's Objectives

- To understand the impact of data transfers on performance when using a GPU as a co-processor
  - speeds and feeds of traditional CPU
  - speeds and feeds when employing a GPU

- To develop a knowledge base for performance tuning for modern GPUs
- To understand multi-GPU interconnect technologies and their role in scaling GPU compute
  - NVLink vs. PCIe for GPU-to-GPU and GPU-to-CPU communication
  - HBM vs. GDDR memory technologies and implications for bandwidth

Review: Canonical CUDA Program Structure

- Global variables declaration
- Kernel functions
  - __global__ void kernelOne(…)

- main () // host code
  - allocate memory space on the device – cudaMalloc(&d_GlblVarPtr, bytes)
  - transfer data from host to device – cudaMemcpy(d_GlblVarPtr, h_Gl…)
  - execution configuration setup
  - kernel call – kernelOne<<<execution configuration>>>(args…);
  - transfer results from device to host – cudaMemcpy(h_GlblVarPtr,…)

  - optional: compare against golden (host computed) solution

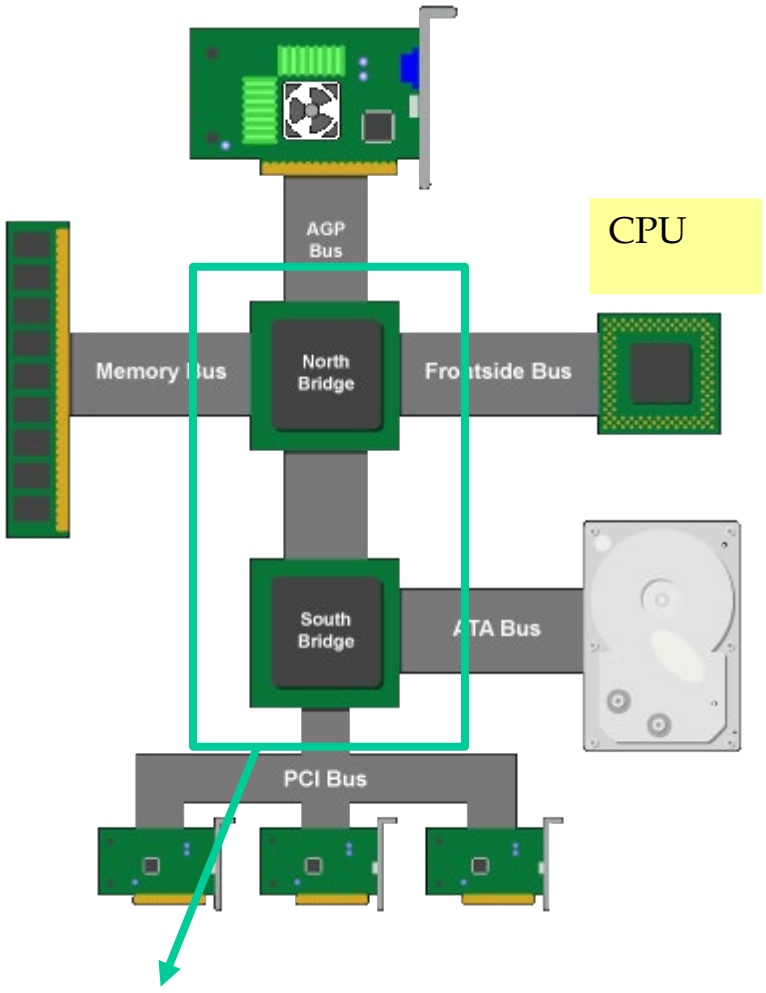
repeat
as needed

Bandwidth: The Gravity of Modern Computer Systems

- **Bandwidth** between key components ultimately **dictates system performance**
  - **Especially for GPUs** processing large amounts of data.
  - Tricks like buffering, reordering, caching can temporarily defy the rules in some cases.
  - Ultimately, performance falls back to what the "speeds and feeds" dictate.
- **Key bandwidth reference points:**
  - CPU main memory (DDR5): ~50–100 GB/s
  - PCIe Gen 4 x16 (CPU↔GPU): ~32 GB/s (bidirectional)
  - NVLink 4 (GPU↔GPU): ~900 GB/s (H100 bidirectional)
  - **GPU HBM3 on-chip memory (H100): ~3.35 TB/s**
  - → PCIe is often the bottleneck in CPU-GPU workloads!

Classic (Historical) PC Architecture

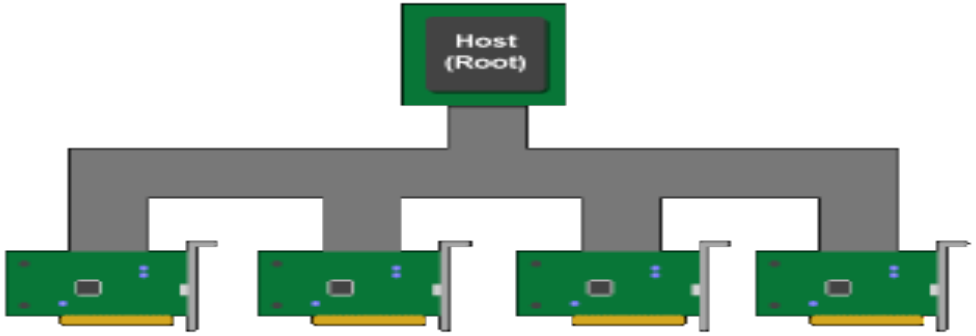
- Northbridge connects 3 components that must communicate at high speed
  - CPU, DRAM, video
  - Video needs first-class access to DRAM
  - Previous NVIDIA cards are connected to AGP, up to 2 GB/s transfers
- Southbridge serves as a concentrator for slower I/O devices



Core Logic Chipset

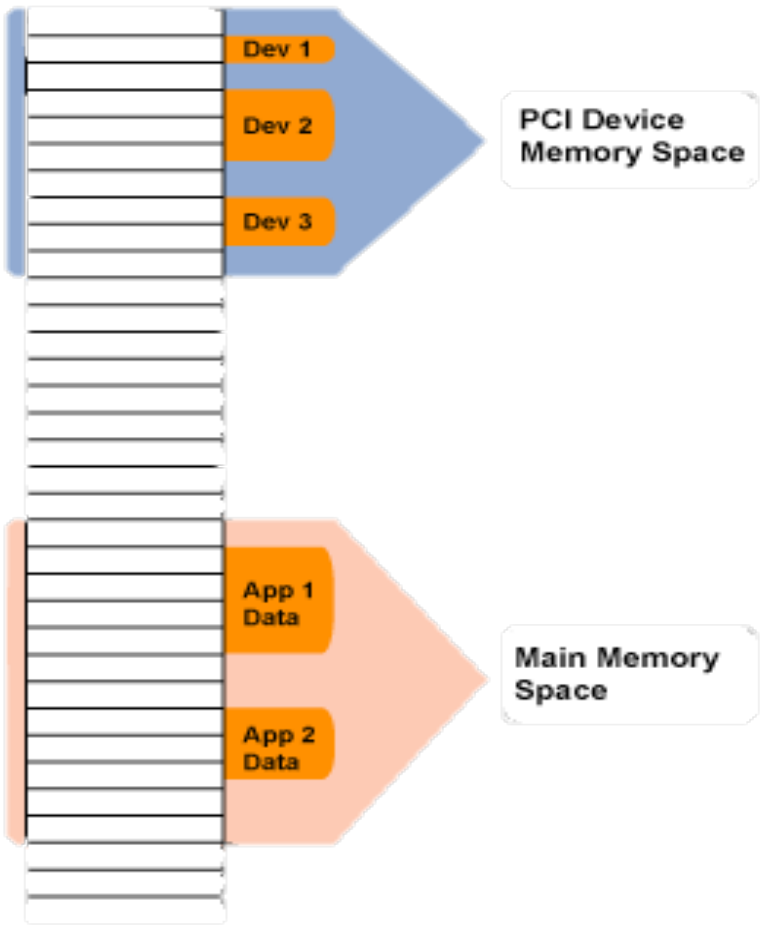
(Original) PCI Bus Specification

- Peripheral Component Interconnect (PCI)
- Connected to the South Bridge
  - Originally 33 MHz, 32-bit wide, 132 MB/second peak transfer rate
  - Later, 66 MHz, 64-bit, 528 MB/second peak
  - Upstream bandwidth remain slow for device (~256MB/s peak)
  - **Shared bus with arbitration**
    - Winner of arbitration becomes bus master and can connect to CPU or DRAM through the southbridge and northbridge



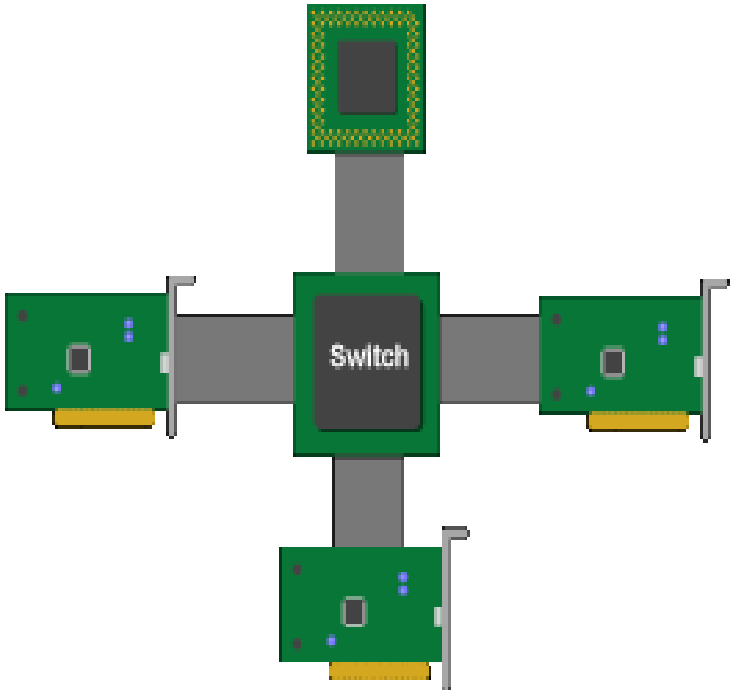
PCI as Memory Mapped I/O

- PCI device registers are mapped into the CPU's physical address space
  - Accessed through loads/stores (kernel mode)
- Addresses are assigned to the PCI devices at boot time
  - All devices listen for their addresses



PCI Express (PCIe)

- **switched, point-to-point connection**
  - each card has dedicated “link” to the central switch, with no arbitration
  - packet switches: messages form virtual channel
  - prioritized packets for QoS (such as for real-time video streaming)

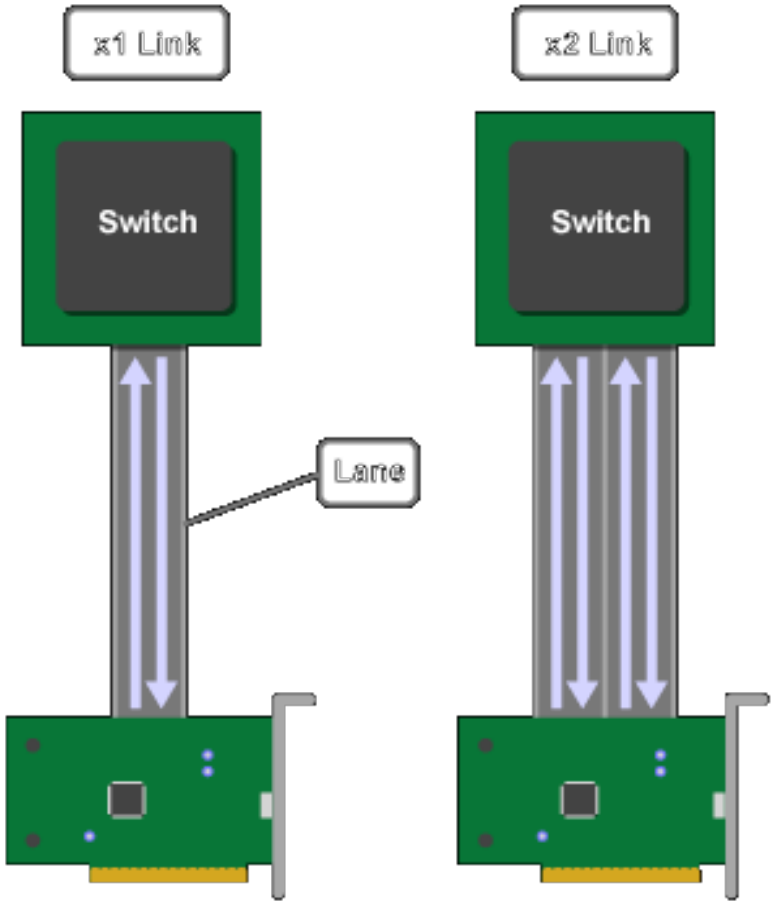


PCIe Generations

- Within a generation, number of lanes in a link can be scaled
  - using distinct physical channels (more bits / wider transfers)
  - ×1, ×2, ×4, ×8, ×16, ×32, …
- Each new generation aims to double the speed
  - PCIe 5.0: 32 GT/s per lane — supported on Intel 12th/13th/14th gen, AMD Ryzen 7000, IBM Power10
  - PCIe 6.0: 64 GT/s per lane, uses PAM4 signaling — now shipping in data center hardware (2023+)
  - x16 bandwidth summary (each direction):
    - Gen 3: ~16 GB/s | Gen 4: ~32 GB/s | Gen 5: ~64 GB/s | Gen 6: ~128 GB/s
  - PCIe 4.0 widely deployed; Gen 3 still common in legacy systems

PCIe Gen 3 Links and Lanes

- Each link consists of one or more lanes
  - Each lane is 1-bit wide (4 wires, each 2-wire pair can transmit 8Gb/s in one direction)
    - 2-wire pair is used for differential signaling Upstream and downstream simultaneous and symmetric
  - Each Link can combine 1, 2, 4, 8, 12, 16 lanes- x1, x2, etc.
- Each byte data is **128b/130b** encoded into 130 bits with equal number of 1's and 0's
  - Thus, the net data rates are 985 MB/s (x1) 1.97 GB/s (x2), 3.94 GB/s (x4), 7.9 GB/s (x8), 15.8 GB/s (x16), each way



Foundation: 8/10 bit encoding

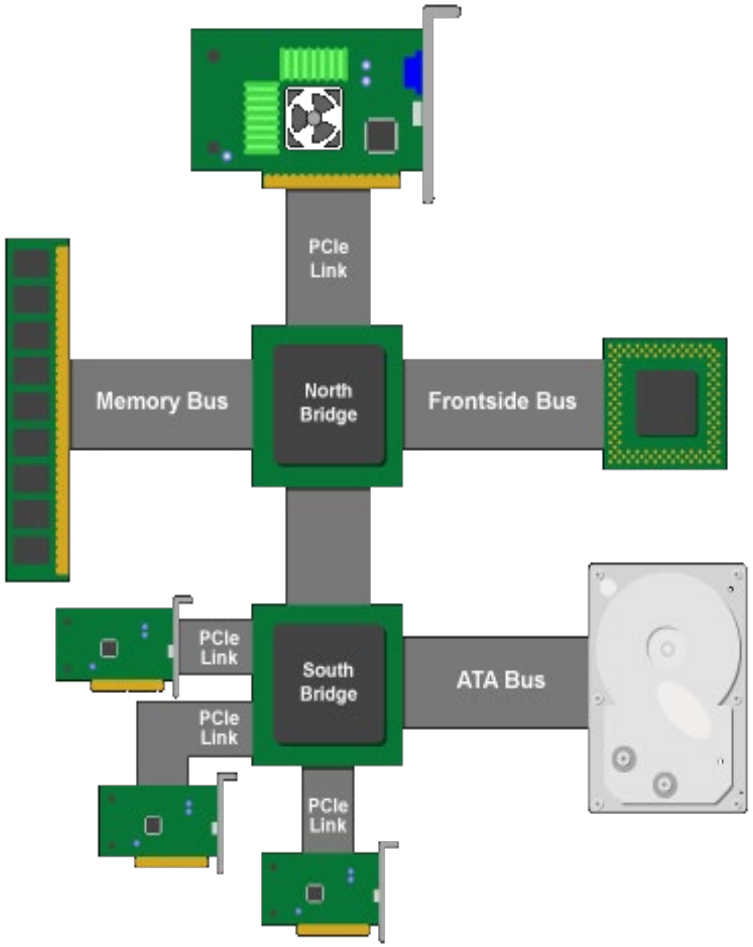
- Goal is to maintain DC balance while have sufficient state transition for clock recovery
- The difference of 1s and 0s in a 20-bit stream should be ≤ 2
- There should be no more than 5 consecutive 1s or 0s in any stream
  - 00000000, 00000111, 11000001 bad
  - 01010101, 11001100 good
- Find 256 good patterns among 1024 total patterns of 10 bits to encode an 8-bit datum
- a 20% overhead

Current: 128/130 bit encoding

- Same goal: maintain DC balance while have sufficient state transition for clock recovery
- 1.5% overhead instead of 20%
- Scrambler function: long runs of 0s, 1s vanishingly small
- Instead of guaranteed run length of 8/10b
- At least one bit shift every 66 bits

Recent PCIe PC Architecture

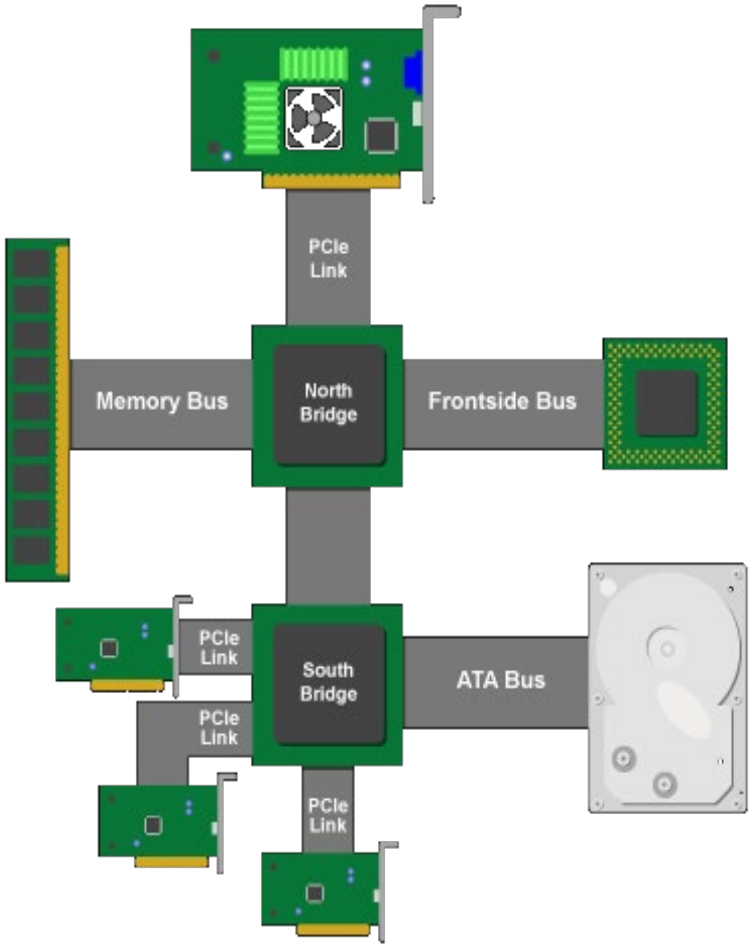
- PCIe forms the interconnect backbone within PC.
- Northbridge and Southbridge are PCIe switches.



Source: Jon Stokes, PCI Express: An Overview (http://arstechnica.com/articles/paedia/hardware/pcie.ars)

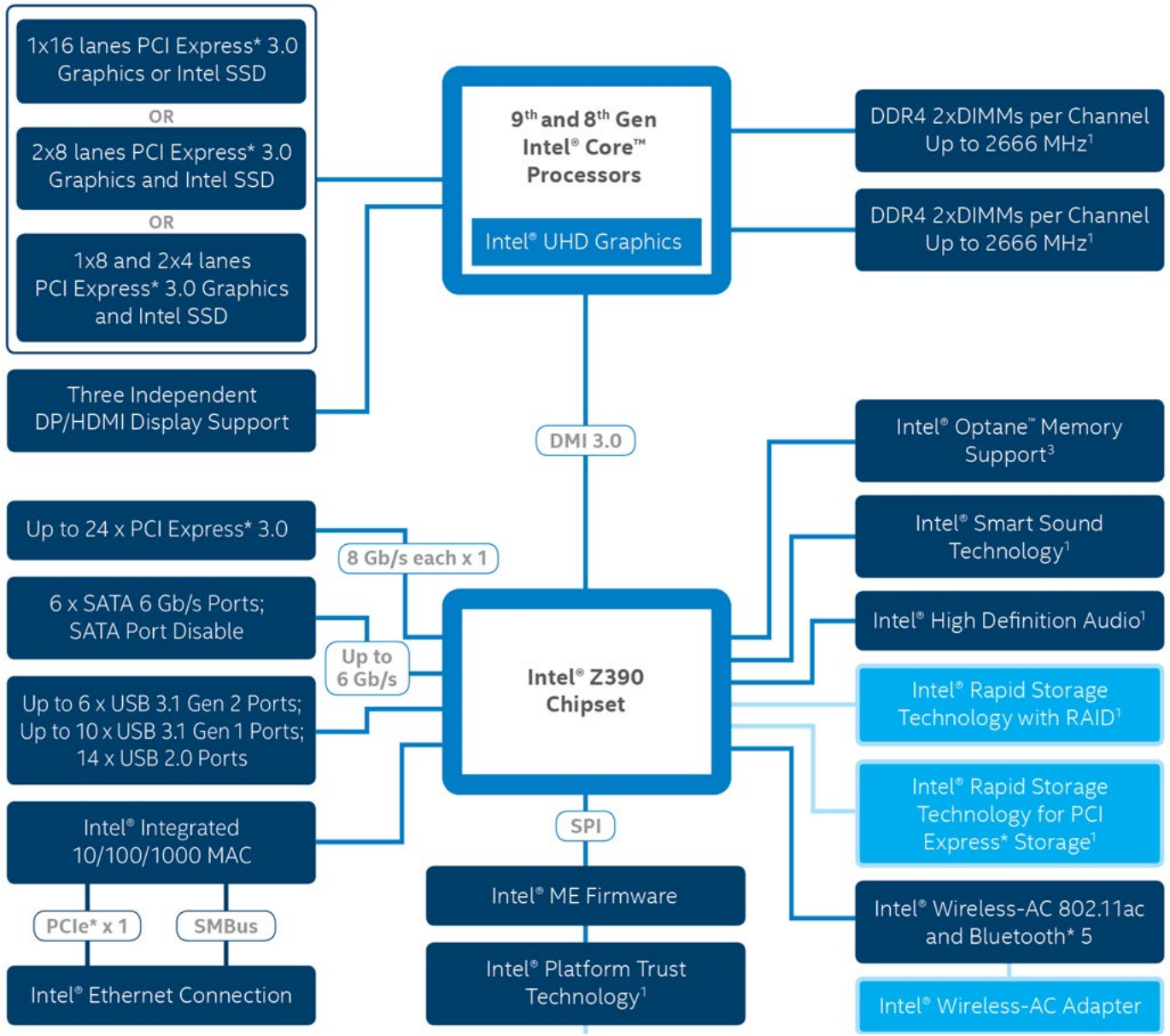
Recent PCIe PC Architecture

- How is PCI supported?
  - Need a PCI-PCIe bridge, which is
  - sometimes included as part of Southbridge, or
  - can add as a separate PCIe I/O card.
- Current systems integrate PCIe controllers directly on chip with CPU.

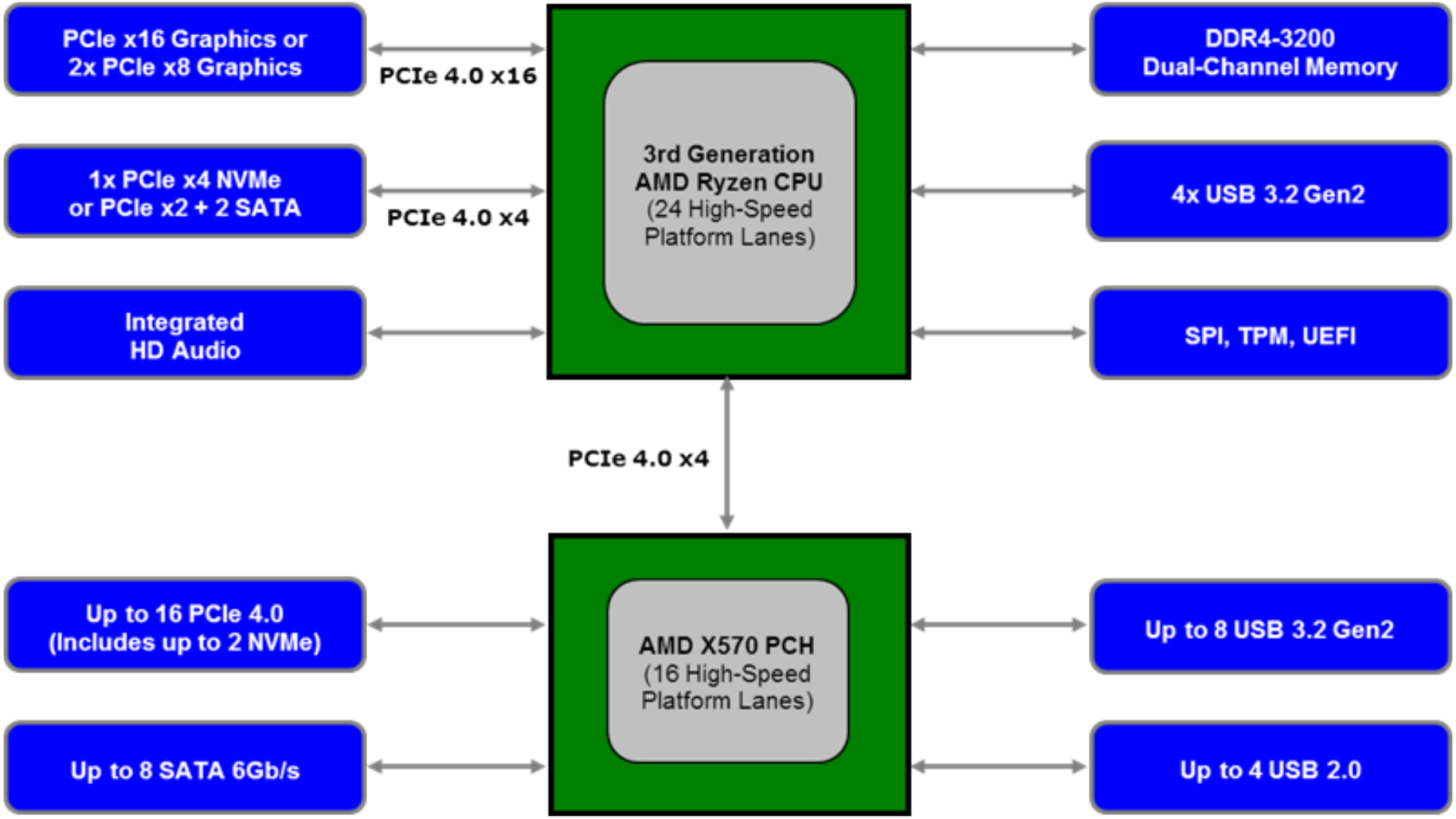


Source: Jon Stokes, PCI Express: An Overview (http://arstechnica.com/articles/paedia/hardware/pcie.ars)

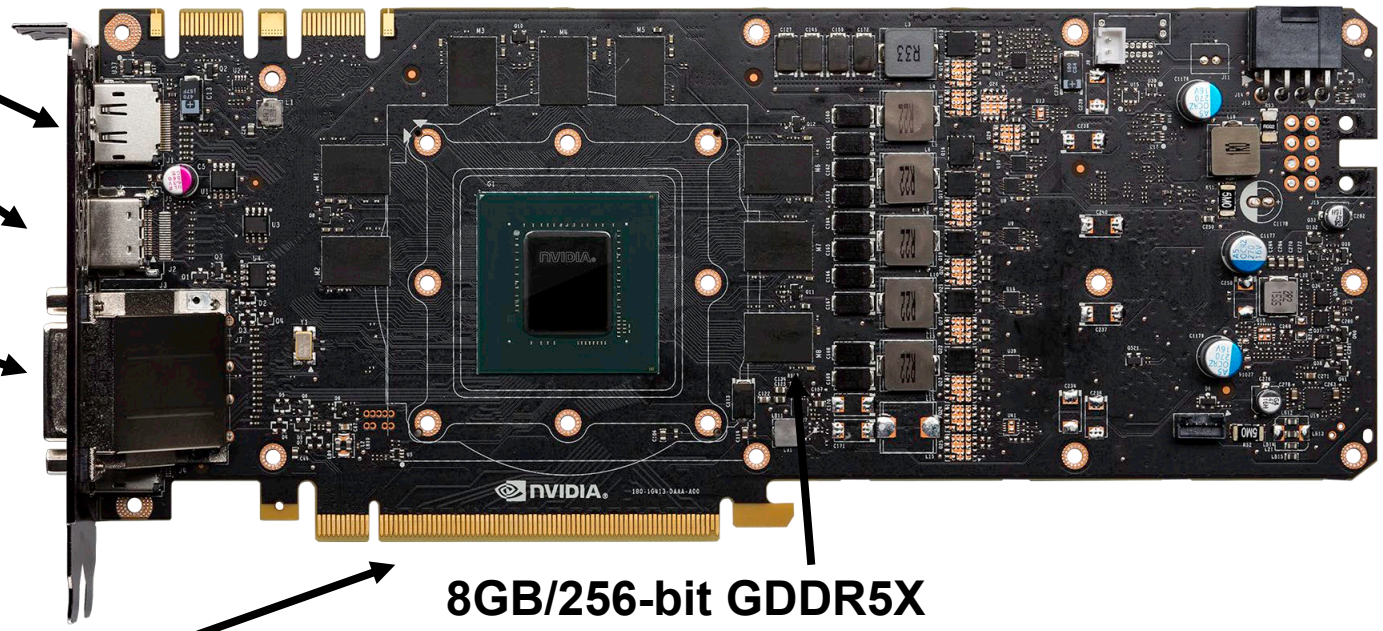
Recent Intel PCIe PC Architecture



Recent AMD PCIe PC Architecture

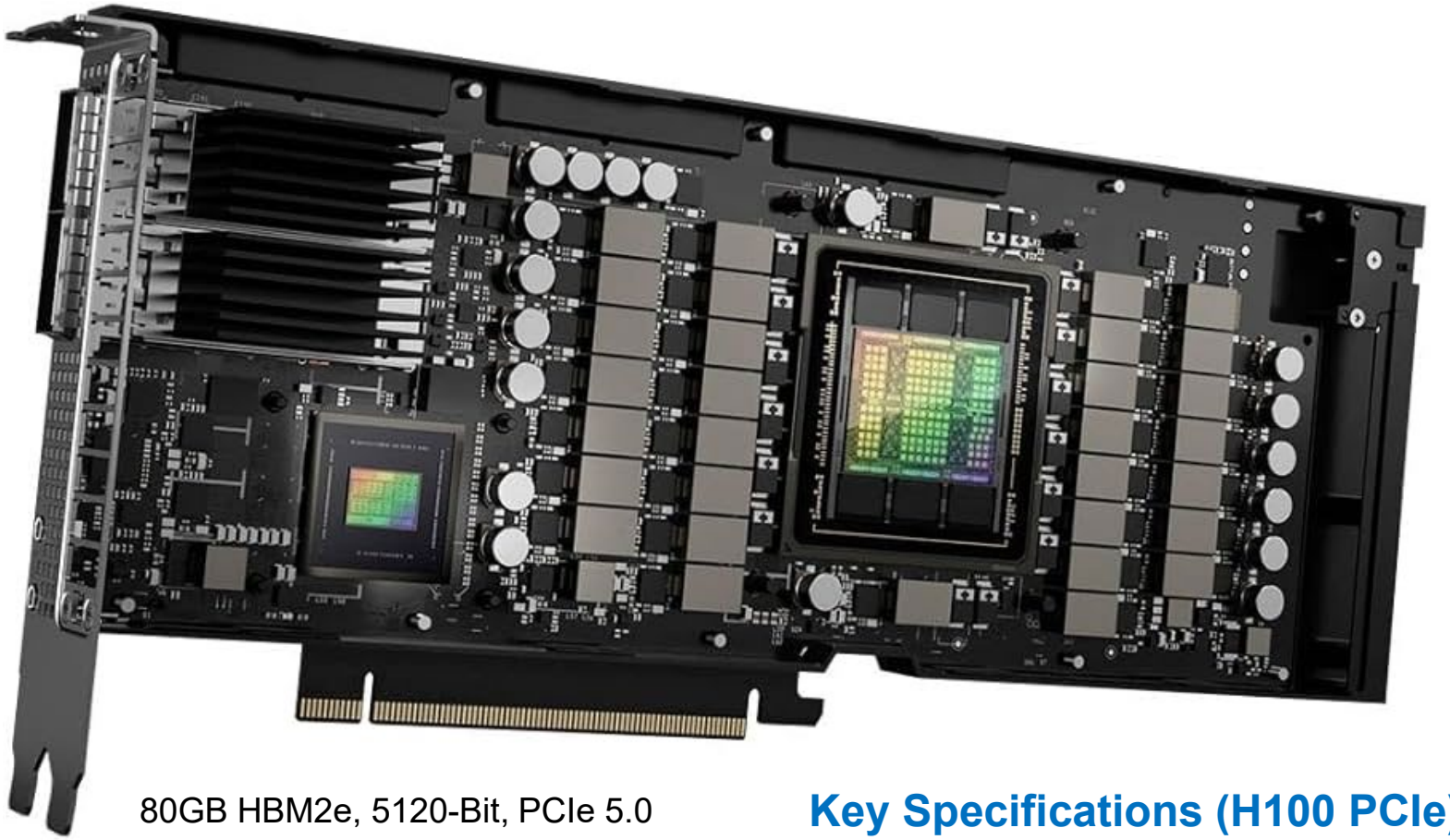


GeForce GTX 1080 (Pascal) GPU Consumer Card Details



8GB/256-bit GDDR5X
1.25 GHz mem clock
2.5 GHz write clock w/ QDR = 10Gb/s/pin
256 bit bus = 320 GB/s
8 pieces of 8Gb (16 Mb x 32 x 16 banks)

NVIDIA H100 (Hopper) PCIe GPU



80GB HBM2e, 5120-Bit, PCIe 5.0

Key Specifications (H100 PCIe):

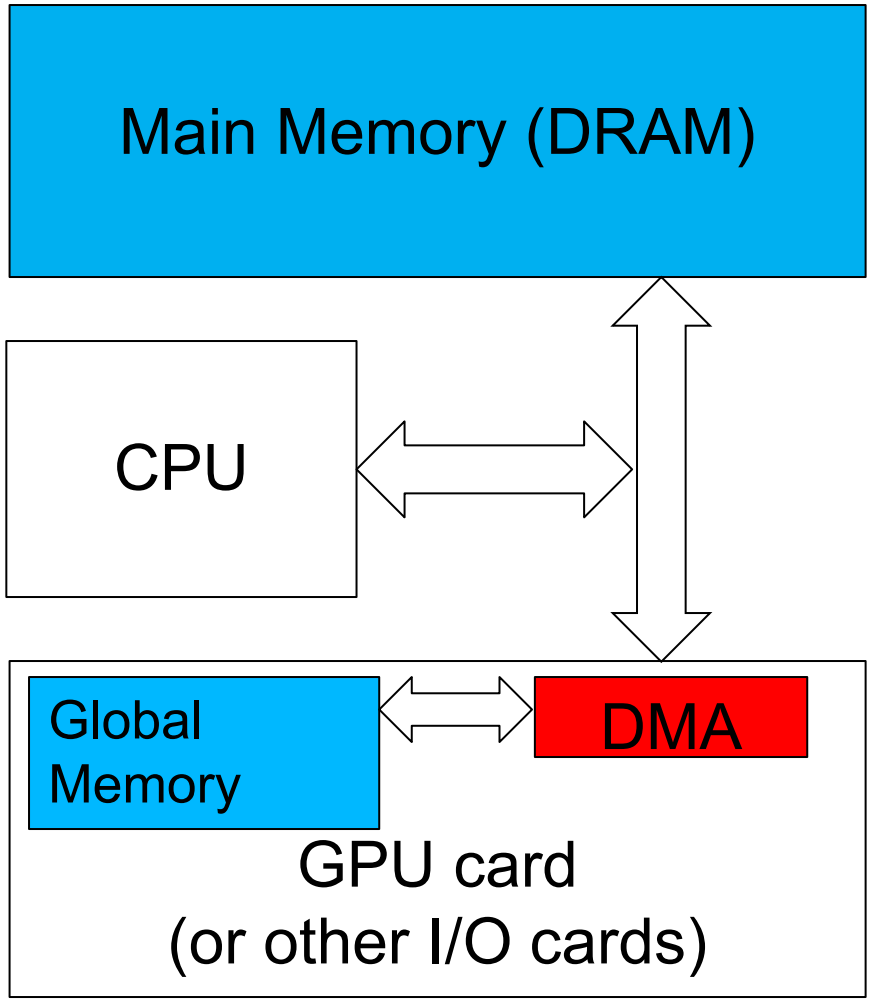
Memory BW: 2,000 GB/s (HBM2e) | SXM5 variant: 3,350 GB/s (HBM3)
NVLink BW: 400 GB/s per GPU (NVLink 4.0) — 12.5x faster than PCIe 5.0 x16
FP16 Tensor Core: up to 1,979 TFLOPS | FP64: 48 TFLOPS
SMs: 114 | TDP: 350 W (PCIe) / 700 W (SXM5)

GPU Memory Technologies: HBM vs. GDDR

- **HBM (High Bandwidth Memory) — used in data center GPUs (A100, H100):**
  - Stacked DRAM dies connected via through-silicon vias (TSVs) on an interposer
  - Extremely wide bus (1024-bit per stack) — trades pin count for bandwidth
  - HBM2e (A100): ~2 TB/s | HBM3 (H100 SXM5): ~3.35 TB/s | lower power per bit
  - Capacity limited (typically 16–80 GB); expensive to manufacture
- **GDDR6 / GDDR6X — used in consumer/workstation GPUs (RTX 4090, A40):**
  - Discrete DRAM chips on PCB; narrower bus (256–384-bit typical)
  - GDDR6 (A40): ~696 GB/s | GDDR6X (RTX 4090): ~1,008 GB/s (PAM4 signaling)
  - Higher capacity per dollar; easier to scale capacity (up to 24+ GB)
- **Key insight: HBM offers 3–5x more bandwidth than GDDR6 but at much higher cost — motivates tiered GPU product lines**

PCIe Data Transfer using DMA

- DMA (Direct Memory Access) is used to fully utilize the bandwidth of an I/O bus
  - DMA uses physical address for source and destination
  - Transfers a number of bytes requested by OS
  - Needs pinned memory



Pinned Memory

- DMA uses physical addresses
- The OS could accidentally page out the data that is being read or written by a DMA and page in another virtual page into the same location
- Pinned memory cannot be paged out
- If a source or destination of a cudaMemcpy in the host memory is not pinned, it needs to be first copied to a pinned memory – extra overhead
- cudaMemcpy is much faster with pinned host memory source or destination

Allocate/Free Pinned Memory (a.k.a. Page Locked Memory)

- cudaHostAlloc()
  - Three parameters
  - Address of pointer to the allocated memory
  - Size of the allocated memory in bytes
  - Option – use cudaHostAllocDefault for now

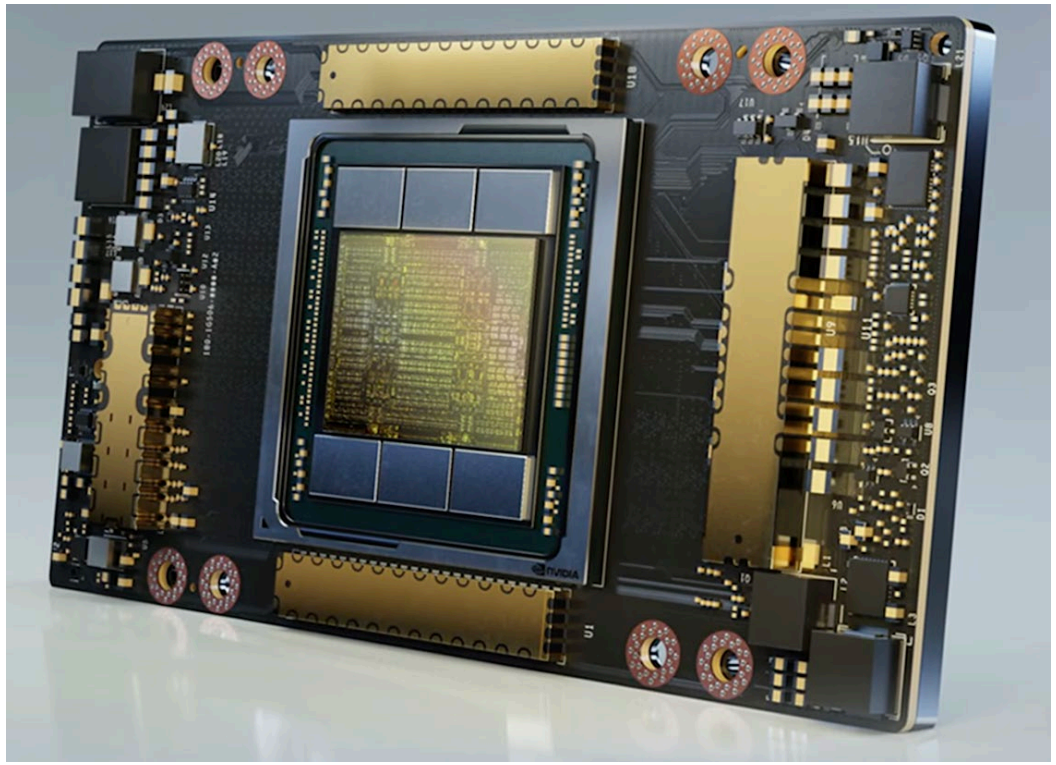
- cudaFreeHost()
  - One parameter
  - Pointer to the memory to be freed

Using Pinned Memory

- Use the allocated memory and its pointer the same way those returned by malloc();
- The only difference is that the allocated memory cannot be paged by the OS
- The cudaMemcpy function should be about 2x faster with pinned memory
- Pinned memory is a limited resource whose over-subscription can have serious consequences

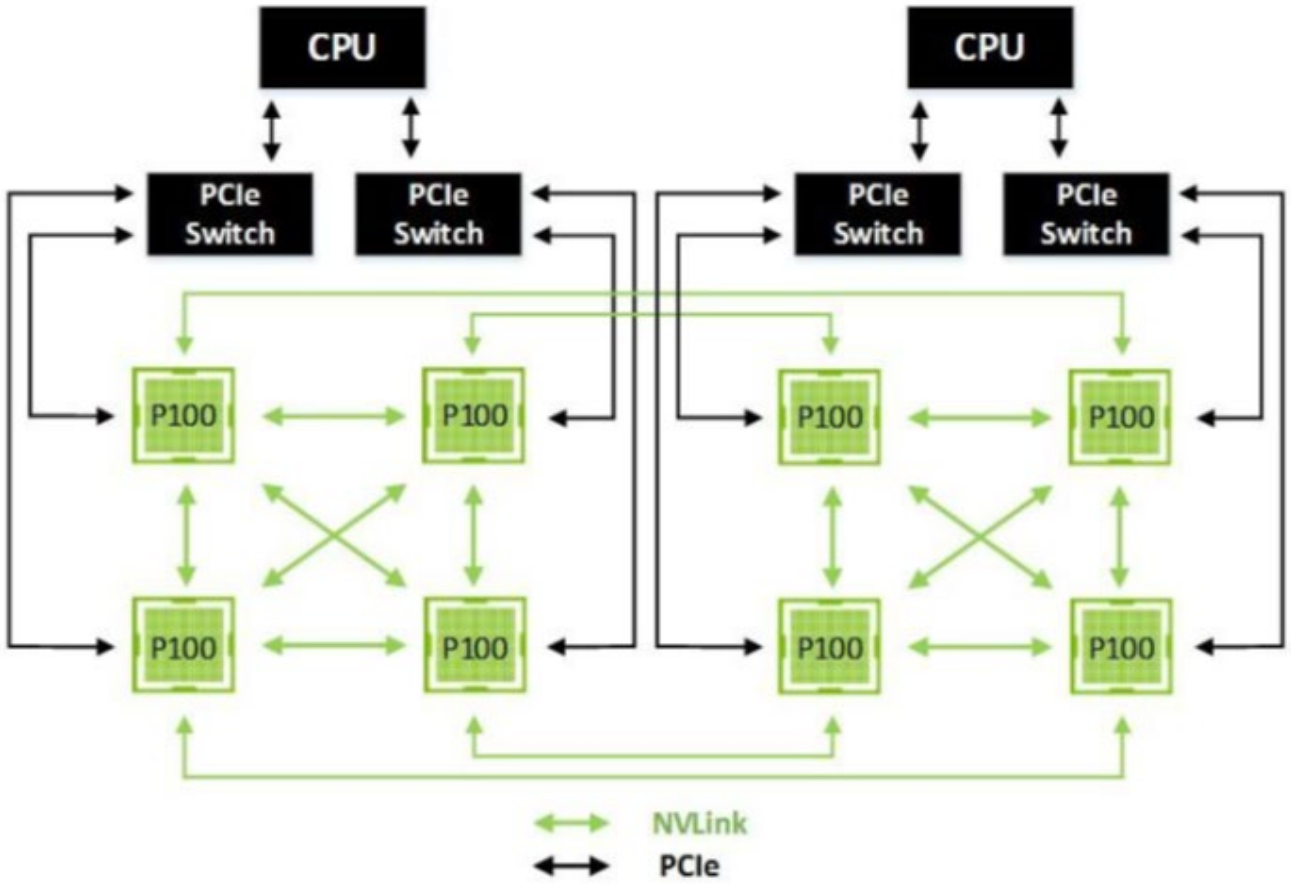
NVIDIA GPU Architectures: Ampere, Hopper & Ada

- **NVIDIA Ampere (A100, 2020):**
  - 3rd-gen Tensor Cores; Multi-Instance GPU (MIG); Structural Sparsity
  - 3rd-gen NVLink: 600 GB/s GPU-to-GPU (~10x PCIe Gen4); HBM2e: 1.6 TB/s
  - Converged acceleration: Ampere + Mellanox ConnectX-6 SmartNIC
- **NVIDIA Hopper (H100, 2022):**
  - 4th-gen Tensor Cores (FP8); Transformer Engine for LLMs
  - HBM3: 3.35 TB/s (SXM5); NVLink 4.0: 900 GB/s bidirectional
  - Grace Hopper (GH200): NVLink-C2C at 900 GB/s — eliminates PCIe bottleneck
- **NVIDIA Ada Lovelace (RTX 4000, 2022):**
  - 3rd-gen RT Cores; 4th-gen Tensor Cores; DLSS 3 Frame Generation
  - GDDR6/GDDR6X memory, up to ~1 TB/s on flagship SKUs



NVLink: Multi-GPU and GPU-CPU Interconnect

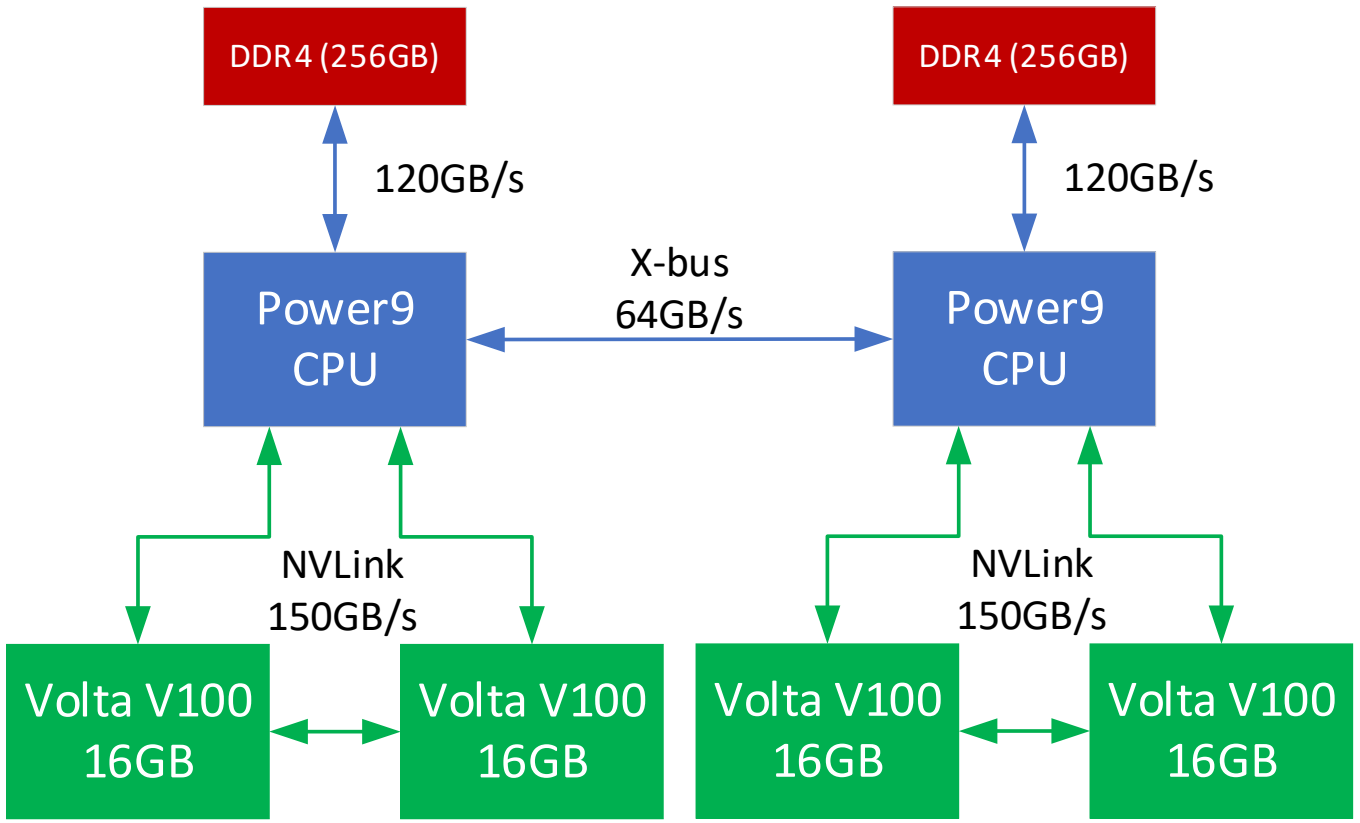
- NVLink Connecting Eight Tesla P100 Accelerators in a Hybrid Cube Mesh Topology.
- GPU-to-GPU data transfers at up to 160 Gigabytes/second of bidirectional bandwidth—5x the bandwidth of PCIe Gen 3 x16.



IBM Power9 System

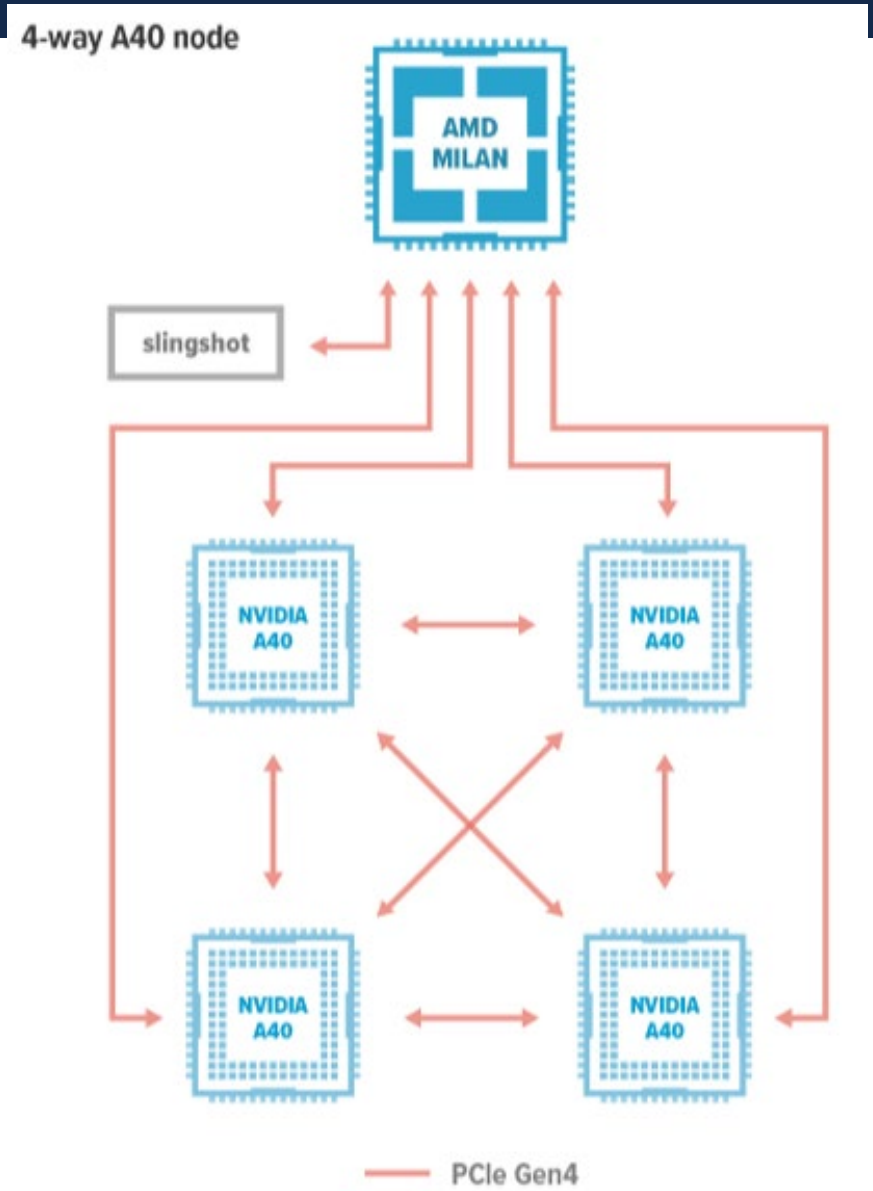
- 2 IBM Power9 CPUs.
- 1 CPU: 10 cores, 80 threads @4.02GHz, 256GB RAM.
- 4 NVIDIA Volta V100 GPUs (16GB).
- GPUs connected using NVLink2.0 Interconnect.

(25GB/s per lane * 6 lanes = 150GB/s)



Delta's NVIDIA A40 GPUs

GPU Memory	48 GB GDDR6 with error-correcting code (ECC)
GPU Memory Bandwidth	696 GB/s
Interconnect	NVIDIA NVLink 112.5 GB/s (bidirectional) PCIe Gen4: 64GB/s
NVLink	2-way low profile (2-slot)
Display Ports	3x DisplayPort 1.4*
Max Power Consumption	300 W
Form Factor	4.4" (H) x 10.5" (L) Dual Slot
Thermal	Passive



Any More Questions?

Things to Read:

Things to Do:
Lab 7

Problem Solving

- Q: The following table gives the SM limits for compute capability (CC) 1.3. Nvidia defines *occupancy* as the ratio of the number of warps per SM for a given kernel configuration divided by the device limit. In a GPU with CC 1.3, which kernel achieves the maximum occupancy?
- A: K2

	CC 1.3	CC 2.0
Max warps per SM	32	48
Max blocks per SM	8	8
Shared memory per SM	16K	48K
Registers per SM	16K	32K
Max block size	512	1024

kernel	blockDim	gridDim	Shared memory per block	Registers per block
K1	160	4096	7K	1K
K2	224	2048	8K	6K
K3	288	1024	10K	9K
K4	96	8192	4K	2K